Structures de données pour l'unification en logique du premier ordre

Rapport en vue de la validation de l'UE Initiation à la recherche

Année universitaire 2025-2026

Etudiants :
Ayath ABOGOURIN
Guillaume POTEL
Matheo VIGNERES

Encadrants :
Julie CAILLER
Marie DUFLOT-KREMER
Engel LEFAUCHEUX

DÉCHARGE DE RESPONSABILITÉS

« L'Université de Lorraine n'entend donner ni approbation ni improbation aux opinions émises dans ce rapport, ces opinions devant être considérées comme propres à leur auteur. »

REMERCIEMENTS

A FAIRE

Sommaire

Introduction 1

I. Généralités 2

1. Logique des propositions 2
2. Logique du premier ordre 2
3. L'unification 3

II. Structures de données et algorithmes . . . . . ??

1. Structures de données représentant un terme
   a. Arbre avec étiquettes
   b. AST (*Abstract Syntax Tree*)
2. Structures de données représentant un ensemble de termes
   a. Liste
   b. Ensemble
   c. Dictionnaire
3. Algorithmes d'unification classiques
   a. Martelli-Montanari
   b. Robinson
4. Le Discrimination Tree

III. Résultats . . . . . ??

1. Protocole de benchmarks
2. Résultats
   a. Martelli-Montanari
   b. Robinson
   c. Discrimination Tree

Conclusion . . . . . ??

Introduction

En informatique, la représentation de l'information est un point clé dans la réalisation de divers projets. Ainsi le choix des **structures de données** représente une étape cruciale pour le bon déroulement de tout projet.

En logique du premier ordre, ce sont les **termes** qui servent de base afin de représenter des expressions symboliques construites à partir de symboles de fonction, de constantes et de variables. Représenter ces termes via la meilleure structure de donnée, la plus adéquate, représente donc un point majeur dans tout travaux portés à ce domaine.

L'**unification** est une opération essentielle dans ce cadre. Elle intervient notamment dans les moteurs d'inférence, les systèmes de réécriture de termes, et les prouveurs logiques. Les différents algorithmes connus d'unification sont parfois adaptés à certaines structures de données précises, il est donc important de comparer ces algorithmes et de diversifier ses choix pour parvenir aux résultats les plus efficaces.

Une structure de données est une représentation permettant d'organiser, traiter et stocker des données de manière efficace dans la mémoire d'un ordinateur. Elle permet un meilleur accès et une manipulation plus efficace des données afin de résoudre un problème spécifique.

Un terme est une expression qui consiste à nommer ou construire un objet en logique. Cela peut être une variable, une constante ou une fonction. Il est en quelque sorte le plus petit composant d'une formule logique.

L'unification consiste à déterminer s'il existe une substitution des variables (c'est-à-dire, une affectations des variables (X, Y , ...) vers des termes (a, b, f(a), ...)) permettant de rendre identiques deux termes.

Ainsi, dans le cadre de ce travail de recherche, nous nous demanderons quelles sont les structures de données les plus adaptées pour l'unification dans la logique du premier ordre. Le choix d'une structure impactant l'efficacité d'un algorithme, nous réfléchirons donc aussi aux meilleurs algorithmes d'unification à utiliser en fonction de la structure de donnée choisie.

Nous commencerons par un rappel sur les différentes notions abordées dans notre sujet. Ensuite nous présenterons plus en détails les différentes structures de données que nous avons utilisées, et nous finirons par la présentation des algorithmes d'unification connus sur lesquels nous avons travaillé.

I. Généralités

D'après nos connaissances, le cours de L3 intitulé Logique des propositions - Logique du premier ordre*, de Jean Lieber et le cours de M1 intitulé* Logiques et Modèles de Calcul*, de Didier Galmiche.*

1. Logique des propositions

2. Logique du premier ordre

Pour représenter des formules dans la *logique du premier ordre*, on se donne un *alphabet* qui servira à les décrire. Cet alphabet est composé des éléments suivants :

— Ensemble de *variables* : $x, y, z, \ldots$

On définit également ce que l'on appelle une *signature*, qui rend notre représentation unique, composée de deux ensembles :

— Ensemble de symboles de *fonctions* : $f, g, h, \ldots$

— Ensemble de symboles de *prédicats* : $P, Q, R, \ldots$

On associe à ces fonctions et prédicats une *arité* (c.-à-d. le nombre d'arguments qu'ils prennent). Parmi les symboles de fonctions, ceux d'arité 0 sont appelés *constantes* :

— Ensemble de symboles de constantes : $a, b, c, \ldots$

Remarque. *Le choix de ces différents symboles est purement arbitraire : on pourrait nommer les variables a, b ou c. Dans un souci d'harmonisation et de standardisation, on utilisera néanmoins les symboles les plus courants dans la littérature.*

Exemple complet.

Considérons la signature suivante :

— Constantes : a, b

— Symbole de fonction : f d'arité 1, g d'arité 2

— Symboles de prédicats : P d'arité 1, Q d'arité 2

On peut alors construire la formule suivante :

$$\forall x \, \big(P(x) \to \exists y \; Q(g(x,\, y),\, f(a))\big)$$

Remarque. *Cette formule se lit : « Pour tout x, si $P(x)$ est vrai, alors il existe un y tel que Q est satisfait par le couple $\big(g(x, y),\, f(a)\big)$ ».*

On peut en détailler la structure :

— x et y sont des *variables* ;

— a est une *constante* (symbole de fonction d'arité 0) ;

— $f(a)$ est un *terme* obtenu en appliquant la fonction f à la constante a ;

— $g(x, y)$ est un *terme* obtenu en appliquant la fonction g aux variables x et y ;

— $P(x)$ et $Q(g(x, y),\, f(a))$ sont des *formules atomiques* (ou atomes), formées en appliquant un symbole de prédicat à des termes ;

— $\forall x$ et $\exists y$ sont des *quantificateurs* liant respectivement x et y ;

— $\to$ est un *connecteur logique* (implication).

3. L'unification

Unification de termes.

On appelle *unification* le processus visant à répondre à la question « Peut-on *substituer* au terme t la variable x ? ».

La *substitution* est l'application qui, à une formule donnée, associe cette même formule avec toutes les occurrences d'un certain terme remplacé, *substitué*, par une variable.

Par exemple, les deux termes t_1 et t_2 suivants :

$$t_1 = f(X,\, g(a)) \qquad \text{et} \qquad t_2 = f(b,\, g(Y))$$

peuvent être unifiés par la substitution $\sigma = \{X \mapsto b,\ Y \mapsto a\}$.

En effet, $\sigma(t_1) = \sigma(t_2) = f(b,\, g(a))$.

Remarque. *L'unification intervient notamment dans les moteurs d'inférence, les systèmes de réécriture de termes, et les prouveurs logiques.*

Unification de prédicats.

L'unification s'étend naturellement aux *atomes*, c'est-à-dire aux formules atomiques de la forme $P(t_1, \ldots, t_n)$. Deux atomes $P(s_1, \ldots, s_n)$ et $P(t_1, \ldots, t_n)$ sont unifiables si et seulement s'ils partagent le même symbole de prédicat P et si leurs arguments respectifs sont unifiables deux à deux.

Ce mécanisme est au cœur de la *règle de résolution* : pour démontrer une formule par réfutation, on cherche à unifier un atome $P(t_1, \ldots, t_n)$ avec sa *négation* $\neg P(s_1, \ldots, s_n)$. Si une substitution σ unifie leurs arguments :

$$\sigma\big(P(t_1, \ldots, t_n)\big) = \sigma\big(P(s_1, \ldots, s_n)\big)$$

alors la résolution produit une *clause vide*, notée $\square$, témoin d'une contradiction — et donc d'une preuve par l'absurde.

Exemple.

Soient les deux clauses $P(X, a)$ et $\neg P(b, Y)$. La substitution $\sigma = \{X \mapsto b,\ Y \mapsto a\}$ unifie leurs atomes :

$$\sigma\big(P(X, a)\big) = P(b, a) = \sigma\big(P(b, Y)\big)$$

La résolution de ces deux clauses produit la clause vide $\square$, concluant à une contradiction.